



DSA Pattern Sheet

Pattern: Gap + Sentinel (Virtual Boundary)

1. Pattern Name

This pattern is known by different names:

- Largest Gap Pattern
- Consecutive Gap Pattern
- Sort + Gap Pattern
- Sentinel Pattern
- Virtual Boundary Pattern
- Interval Gap Pattern

All are variations of the same idea.

2. One-Line Definition

Instead of checking every element,

find the **empty space between important positions**.

3. Recognition Keywords

Whenever a problem contains words like

- blocked
- occupied
- reserved
- forbidden
- unavailable
- enemy
- obstacle
- traffic signal
- stone
- wall
- cut
- interval
- partition
- divide

- consecutive
- continuous
- free space
- empty area

Immediately ask

"Is this actually asking me to find the largest gap?"

4. Brain Trigger

Whenever you read

Some positions are blocked.

Your brain should automatically think

Blocked Positions

↓

Sort Them

↓

Find Gaps

↓

Answer

5. Generic Pattern

Suppose

Important Positions

5

10

15

Instead of thinking

```
1 2 3 4 5 6 7 8 ...
```

Think

```
0
5
10
15
END
```

Only gaps matter.

6. General Algorithm

Step 1

Collect important positions.

```
blocked
reserved
occupied
cuts
signals
```

↓

Step 2

Sort them.

↓

Step 3

Add virtual boundaries.

↓

Step 4

Compute every gap.

↓

Step 5

Return

Maximum

or

Minimum

depending on problem.

7. Universal Template

```
positions.sort()

positions = [LEFT] + positions + [RIGHT]

answer = 0

for i in range(1, len(positions)):

    gap = positions[i] - positions[i-1] - OFFSET

    answer = max(answer, gap)

return answer
```

Notice

Only

LEFT

RIGHT

OFFSET

change.

Everything else stays same.

8. How to Identify in Interviews

Question 1

Do actual values matter?

or

Only positions matter?

Example

Blocked rows.

Answer

Only row numbers matter.

Pattern found.

Question 2

Can I ignore everything between two obstacles?

If YES

Gap Pattern.

Question 3

Can I sort first?

If YES

Gap Pattern becomes easier.

Question 4

Can I transform

Cells

↓

Rows

↓

Positions

Pattern detected.

Question 5

Are there edge cases at beginning/end?

If YES

Think

Sentinel.

9. Sentinel Pattern

Definition

Sentinel means

Artificial value

added

to remove edge cases.

Example

Rows

1

2

3

4

5

Blocked

3

Beginning has no blocked row.

Instead of writing

special code,

add

0

Similarly

End

n+1

Now every case becomes identical.

10. Why Sentinels?

Without sentinel

Need

Beginning

Middle

End

Three cases.

With sentinel

Only

One Formula.

Huge simplification.

11. How to Think Like an Interviewer

Brute Force

↓

Repeated Work?

↓

Can I ignore unnecessary information?

↓

Can I transform problem?

↓

Can I use positions instead of values?

↓

Can I sort?

↓

Can I use gaps?

↓

Can I remove edge cases?

↓

Done.

12. Problem Transformation Pattern

Never solve

Original Problem.

Transform it.

Example

Original

Largest Rectangle.

↓

Transformation

Largest Row Gap

×

Largest Column Gap.

Much easier.

13. General Thinking Framework

Read problem.

↓

Ignore constraints.

↓

Build brute force.

↓

Ask

"What am I repeatedly checking?"

↓

Ask

"Can I avoid checking elements?"

↓

Ask

"What property determines everything?"

↓

Solve using property.

14. Questions To Ask Yourself

Whenever stuck.

Ask these.

①

What changes after every operation?

②

What information actually matters?

③

Am I solving at the wrong level?

Example

Cells

↓

Rows

↓

Positions

④

Can I remove repeated work?

⑤

Can sorting help?

⑥

Can I convert into intervals?

⑦

Can I calculate gaps?

⑧

Can fake boundaries remove edge cases?

⑨

Can one formula solve every case?

⑩

Am I solving the transformed problem

or

the original problem?

15. Similar Interview Problems

This pattern appears in

Arrays

Largest consecutive free indices

Largest gap between numbers

Maximum empty seats

Maximum free parking slots

Reserved seats

Meeting rooms

Memory allocation

CPU scheduling

Disk allocation

Strings

Longest gap between characters

Longest distance between vowels

Geometry

River crossing

Stepping stones

Road construction

Street lights

Road dividers

Grid

Blocked rows

Blocked columns

Enemy attacks

Laser beams

Firewall grids

Intervals

Merge intervals

Insert interval

Maximum interval

Minimum interval

Calendar problems

16. Real World Examples

Airport Runways

Occupied runways

↓

Largest free runway.

Parking Lots

Occupied slots

↓

Largest consecutive parking space.

Memory Allocation

Allocated blocks

↓

Largest free memory block.

Road Traffic

Traffic signals

↓

Longest road without signal.

Hotel

Occupied rooms

↓

Largest consecutive empty rooms.

Train Reservation

Booked seats

↓

Largest available block.

Warehouse

Occupied shelves

↓

Largest free shelf segment.

17. Mistakes Beginners Make

✗ Build entire grid.

Instead

Use rows/columns.

✗ Check every cell.

Instead

Check gaps.

✗ Handle first and last separately.

Instead

Use sentinels.

✗ Memorize formula.

Instead

Understand

Walls

↓

Free Region

↓

Gap.

18. Cheat Sheet

Brute Force

↓

Repeated Work

↓

Higher Abstraction

↓

Transform Problem

↓

Sort

↓

Sentinel

↓

Gap

↓

Answer

19. One-Line Memory Rules

Rule 1

Obstacles create gaps.

Rule 2

Sort before finding gaps.

Rule 3

Sentinels remove edge cases.

Rule 4

Don't inspect every element.

Inspect boundaries.

Rule 5

Transform the problem before optimizing.

Rule 6

Higher abstraction usually gives lower complexity.

Cells

↓

Rows

↓

Positions

20. The Golden Interview Template

Whenever you solve ANY problem.

Step 1

Understand.

↓

Step 2

Brute Force.

↓

Step 3

Find repeated work.

↓

Step 4

Find hidden property.

↓

Step 5

Transform problem.

↓

Step 6

Choose suitable pattern

- Prefix Sum
- Two Pointers
- DP
- Greedy
- Binary Search
- Gap Pattern
- Graph
- Stack
- Queue

↓

Step 7

Remove edge cases.

↓

Step 8

Code.

↓

Step 9

Dry Run.

↓

Step 10

Complexity Analysis.



Ultimate Lesson from This Problem

Optimization is rarely about reducing loops. It is about changing the way you look at the problem.

Brute Force looked at **cells**.

Optimized solution looked at **gaps between blocked positions**.

That change of perspective reduced the complexity from **$O(n^3m^3)$** to **$O(k \log k)$** .